

INFORME TÉCNICO

Refactor de Interfaz de Usuario

IAW-Commerce · Next.js + Tailwind CSS

Guía de Estudio — Next.js 14 · Tailwind CSS v3 · App Router · React Server Components

PARTE 1 — REFACTOR DE INTERFAZ (UI MODERNIZATION)

1.1 Motivación y criterios de diseño

El objetivo fue modernizar la interfaz partiendo de una base funcional pero visualmente inconsistente. A continuación se listan los problemas identificados antes del refactor y los criterios adoptados para resolverlos.

Problemas detectados

- **Elementos HTML inválidos:** `<Button>` de Headless UI conteniendo `<Link>`, y `<Link>` conteniendo `<button>`. Esto genera nodos interactivos anidados, viola la especificación HTML5 y produce comportamiento impredecible en lectores de pantalla y eventos de click.
- **Paleta de colores mezclada:** botones con bg-blue-500, bg-blue-600, bg-blue-400 sin un color de marca definido.
- **Falta de jerarquía visual:** cards sin sombra, formularios sin card contenedora, navbar sin height fija.
- **Inconsistencia en bordes:** algunos elementos con rounded, otros con rounded-lg, otros sin borde.

i Nota: Los nodos interactivos anidados (`<button>` dentro de `<a>` o viceversa) son ilegales según la especificación HTML5. El navegador los intenta corregir automáticamente, pero el resultado es impredecible: el click puede dispararse dos veces o no propagarse correctamente. Además, los lectores de pantalla (NVDA, VoiceOver) reportan ambos roles, generando confusión en usuarios con discapacidad visual.

Criterios adoptados para el refactor

- **Color de marca único:** #004AAD (hover: #003d8f).
- **Elementos interactivos limpios:** todos son `<Link>` o `<button>` directos, sin anidamiento.
- **Bordes redondeados consistentes:** rounded-xl para inputs y botones primarios, rounded-2xl para cards y paneles.
- **Sombra coherente:** shadow-sm en cards en reposo, shadow-lg en hover.
- **Fondo de la aplicación:** gray-50 o gray-100 (nunca blanco puro en el contenedor raíz).

1.2 Sistema de colores y tokens de marca

Antes del refactor los colores estaban dispersos en cada componente. Se unificaron en los siguientes tokens semánticos:

Token semántico	Valor Tailwind / HEX	Uso
Color primario	bg-[#004AAD]	CTA principal, botones de acción
Color primario hover	bg-[#003d8f]	Estado :hover del primario
Acento / logo	text-blue-400	Logo IAW, highlights
Fondo de página	bg-gray-50 / bg-gray-100	Contenedor raíz
Fondo card / panel	bg-white	Cards, modales, formularios
Texto principal	text-gray-900	Títulos, cuerpo primario
Texto secundario	text-gray-600/500	Subtítulos, metadata
Texto deshabilitado	text-gray-400	Placeholders, estados inactivos
Borde estándar	border-gray-100/200	Cards, inputs
NavBar fondo	bg-gray-900	Barra de navegación
Hero degradado	from-gray-900 via-[#003580] to-[#004AAD]	Sección hero de landing

i Nota: El color #004AAD no fue elegido arbitrariamente: ya existía como `themeColor` en el manifest PWA (`src/app/layout.tsx` → `viewport.themeColor`). El refactor simplemente alineó la UI con la identidad ya definida para la app instalable.

¿Por qué usar tokens en lugar de clases directas?

Usar clases arbitrarias como `bg-[#004AAD]` en lugar de definir un color personalizado en `tailwind.config.ts` es un anti-patrón para proyectos grandes. La forma correcta es:

```
// tailwind.config.ts
theme: {
  extend: {
    colors: {
      brand: {
        DEFAULT: '#004AAD',
        dark: '#003d8f',
      }
    }
  }
}
```

Con esto, se puede escribir `bg-brand` y `hover:bg-brand-dark`, que son más legibles, más fáciles de mantener y soportan el motor de purga de Tailwind correctamente.

1.3 NavBar y botones de navegación

Archivo: `src/app/ui/shared/NavBar.tsx`

Problema original

- La raíz era un `<div>`, lo que no es semánticamente correcto para una barra de navegación.
- `<HomeButton>` internamente tenía `<Button><Link>`, generando nodo interactivo anidado.
- Sin height fija: el navbar crecía según su contenido.

Solución aplicada

- Se cambió el `<div>` raíz por `<nav aria-label="Navegación principal">`. Esto expone el landmark de navegación a tecnologías de asistencia.
- Se eliminó la importación de `HomeButton` y se reemplazó por un `<Link>` inline dentro del `nav`.
- Se fijó height con `h-14 (56px)`. Se usó `sticky top-0 z-50` para que el navbar acompañe el scroll.

Técnica link-as-button

Consiste en aplicar estilos visuales de botón (`padding`, `background`, `border-radius`, `font-weight`) a un elemento `<Link>` o `<a>`. El elemento sigue siendo semánticamente un enlace, pero se ve como un botón. Esto es correcto desde el punto de vista de accesibilidad siempre que el destino sea una URL, no una acción.

```
// ✅ CORRECTO: link que parece botón
<Link
  href="/login"
  className="px-4 py-2 bg-[#004AAD] text-white rounded-xl font-medium
  hover:bg-[#003d8f]"
>
  Iniciar sesión
</Link>
```

```
// ✅ CORRECTO: botón de acción (no navega)
<button onClick={() => signOut()} className="px-4 py-2 ..."
  Cerrar sesión
</button>
```

```
// ❌ INCORRECTO: nodo interactivo anidado
<Button> { /* Headless UI: renderiza como <button> */ }
  <Link href="/login">Iniciar sesión</Link>
</Button>
```

Regla de oro para decidir entre `<Link>` y `<button>`

Usar <code><Link></code>	Usar <code><button></code>
La acción navega a una URL	La acción no navega (muta estado, llama API)
Puede abrirse en nueva pestaña	No tiene URL destino
El historial del navegador debe registrarlo	<code>signOut</code> , <code>addToCart</code> , <code>openModal</code>

1.4 Landing Page (página raíz)

Archivo: `src/app/page.tsx`

- Se eliminó el patrón incorrecto `<Button><Link>`.
- El hero section tiene degradado `bg-gradient-to-br from-gray-900 via-[#003580] to-[#004AAD]`.
- CTA principal: `<Link>` con estilos de botón blanco (`bg-white text-[#004AAD]`) para contraste sobre el fondo oscuro.
- Sección de cierre con fondo `bg-gray-900` y CTA `bg-[#004AAD]`. Consistente con la paleta de marca.
- `<main>` semántico envuelve las secciones de contenido.
- Padding horizontal explícito en cada sección con `px-4` para evitar overflow horizontal en mobile.

Contraste WCAG en el hero

El color blanco (`#FFFFFF`) sobre `#004AAD` tiene un ratio de contraste de 7.5:1, que supera el mínimo AA de WCAG 2.1 (4.5:1 para texto normal) y cumple el nivel AAA (7:1). Esto es importante para garantizar legibilidad para usuarios con baja visión.

1.5 Tarjetas de producto (Cards)

Archivo: `src/app/ui/dashboard/Cards.tsx`

Cambios aplicados

- **Contenedor:** `rounded-2xl border border-gray-100 bg-white shadow-sm transition-shadow hover:shadow-lg`
- **Imagen con fill + object-cover:** mantiene el aspect ratio rellenando el contenedor sin distorsión.
- **Jerarquía de botones:** "Ver detalles" (outline) vs "Agregar al carrito" (solid). La acción primaria tiene mayor peso visual.

El atributo `sizes` en `next/image`

El atributo `sizes` informa al navegador qué porcentaje del viewport va a ocupar la imagen en cada breakpoint, para que descargue la versión más liviana del srcset generado automáticamente por Next.js:

```
sizes="(max-width: 768px) 100vw,  
      (max-width: 1200px) 50vw,  
      33vw"
```

Si omitimos sizes, el navegador asume 100vw y descarga siempre la imagen más grande, desperdiciando ancho de banda en desktop.

fill vs width/height en next/image

Prop	Cuándo usarlo	Requisito del contenedor
fill	El contenedor tiene dimensiones definidas por CSS	position: relative + dimensiones explícitas
width + height	Se conoce el tamaño exacto en px	Ninguno especial

1.6 Barra de búsqueda (SearchBar)

Archivo: src/app/ui/dashboard/SearchBar.tsx

- Ícono <MagnifyingGlassIcon> de Heroicons integrado en el botón de búsqueda.
- El campo de texto y el botón comparten un único contenedor con borde, uniéndolos visualmente.
- Se soporta búsqueda con Enter mediante onKeyDown.

La pseudo-clase focus-within

focus-within aplica estilos al elemento padre cuando cualquier descendiente tiene foco. Es diferente de focus, que sólo aplica al elemento que recibe el foco directamente:

```
// El contenedor cambia de borde cuando el input interno tiene foco
<div className="border border-gray-200 rounded-xl
  focus-within:border-[#004AAD]
  focus-within:ring-1 focus-within:ring-[#004AAD]">
  <input type="text" className="focus:outline-none" />
  <button>Buscar</button>
</div>
```

Esto es superior a poner focus en cada elemento hijo, porque da una experiencia de foco unificada. Al tabular al input o al botón, el contenedor entero reacciona.

i Nota: El atributo correcto es onKeyDown (no onKeyPress, que fue deprecado en HTML5). Para detectar Enter: if (e.key === 'Enter') { handleSearch(); }

1.7 Paginación

Archivo: src/app/ui/dashboard/Pagination.tsx

- Links de página con estilos de botón: border border-gray-300 rounded-lg px-3 py-2.

- Página activa destacada: `bg-[#004AAD] text-white font-bold`.
- Textos traducidos: 'Anterior' / 'Siguiente'.
- Se agregaron íconos `<ChevronLeftIcon>` y `<ChevronRightIcon>`.
- El texto 'Anterior'/'Siguiente' usa `hidden sm:inline` → solo ícono en mobile.

Paginación con URL params en Next.js App Router

En el App Router, la paginación se implementa con `searchParams`, lo que permite que las URLs sean compartibles y que el estado de paginación sea parte de la URL:

```
// src/app/dashboard/page.tsx
export default async function Page({
  searchParams,
}): {
  searchParams: { page?: string; query?: string };
}() {
  const page = Number(searchParams.page) || 1;
  const query = searchParams.query || '';
  const products = await fetchProducts({ page, query });
  return <ProductListDashboard products={products} />;
}
```

El componente de paginación simplemente genera `<Link>` con los `searchParams` actualizados, sin necesidad de `useState`.

1.8 Diálogo 'Agregar al carrito'

Archivo: `src/app/ui/dashboard/AddToCartButton.tsx`

Cambios principales

- **Backdrop oscuro:** `bg-black/50` detrás del panel del diálogo. Sin esto, el usuario no percibe que hay un modal activo.
- **Animación de escala:** `scale-95` → `scale-100` (entrada) y `scale-100` → `scale-95` (salida).
- **CheckCircleIcon:** refuerzo visual de confirmación (feedback positivo de color verde).
- **Botones diferenciados:** 'Entendido' (`bg-blue-100`) vs 'Ir al carrito' (`bg-gray-100`).

Implementación del backdrop con Headless UI Transition

```
// TransitionChild para el backdrop
<TransitionChild
  enter="ease-out duration-300"
  enterFrom="opacity-0"
  enterTo="opacity-100"
  leave="ease-in duration-200"
  leaveFrom="opacity-100"
  leaveTo="opacity-0"
/>
```

```
<div className="fixed inset-0 bg-black/50" />
</TransitionChild>
```

```
// TransitionChild para el panel del modal
<TransitionChild
  enter="ease-out duration-300"
  enterFrom="opacity-0 scale-95"
  enterTo="opacity-100 scale-100"
  leave="ease-in duration-200"
  leaveFrom="opacity-100 scale-100"
  leaveTo="opacity-0 scale-95"
>
  <DialogPanel className="...">
    {/* contenido */}
  </DialogPanel>
</TransitionChild>
```

1.9 CartCard (ítem del carrito)

Archivo: `src/app/ui/cart/CartCard.tsx`

- **Imagen:** relative h-20 w-20 overflow-hidden rounded-xl con `<Image fill object-cover>`.
- **Botones de cantidad:** botones circulares con `<MinusIcon>` y `<PlusIcon>`. border border-gray-300 con hover:bg-gray-100.
- **Botón eliminar:** circular bg-red-500 hover:bg-red-600 con `<TrashIcon>`. El rojo señala acción destructiva.
- **aria-label en cada botón:** "Aumentar cantidad", "Disminuir cantidad", "Quitar producto".

i Nota: Los aria-labels son obligatorios en botones que solo muestran íconos (sin texto visible). Sin ellos, un lector de pantalla anunciaría "botón" sin contexto. El aria-label reemplaza el texto visible para tecnologías de asistencia.

1.10 Formularios Login y Register

Archivos: `src/app/login/LoginForm.tsx`, `src/app/register/page.tsx`

Layout del panel

- Panel izquierdo: bg-gray-50, formulario centrado con flexbox.
- Panel derecho (solo desktop lg): degradado de marca with from-gray-900 via-[#003580] to-[#004AAD] con logo y slogan.
- El panel derecho usa hidden lg:flex → invisible en mobile y tablet.

Formulario

- **Contenedor:** rounded-2xl shadow-sm border border-gray-100 bg-white.
- **Inputs:** bg-gray-50 border border-gray-200 rounded-xl con focus:ring-2 focus:ring-[#004AAD].
- **Botón submit:** w-full bg-[#004AAD] rounded-xl → ocupa todo el ancho como CTA principal.

Focus ring personalizado

Por defecto, Tailwind CSS 3 aplica focus:ring-2 focus:ring-blue-500 outline-none. Para usar el color de marca:

```
// Reemplazar el focus ring de Tailwind con el color de marca
className="... focus:outline-none focus:ring-2 focus:ring-[#004AAD]
focus:ring-offset-1"
```

ring-offset-1 agrega un espacio blanco entre el input y el anillo, mejorando la legibilidad visual del foco en fondos de color similar.

1.11 Detalle de producto (ProductDetails)

Archivo: src/app/ui/dashboard/ProductDetails.tsx

Solución

- **Botón de volver:** <Link> directo con <ChevronLeftIcon> y texto 'Volver al catálogo'.
- **Card contenedora:** bg-white rounded-2xl border border-gray-100 shadow-sm overflow-hidden.
- **Imagen:** div con relative aspect-square (en sm+) + <Image fill object-cover>.
- **Layout interior:** flex flex-col md:flex-row. En mobile apilado, en desktop lado a lado.
- **Precio destacado:** text-3xl font-extrabold text-[#004AAD]. El precio es el segundo dato mas importante.

i Nota: overflow-hidden en el contenedor padre es necesario cuando se usan border-radius. Sin él, la imagen rectangular desborda los bordes redondeados de la card, especialmente visible en las esquinas superiores donde la imagen comienza.

1.12 Carrito — lista y resumen (ProductList)

Archivo: src/app/ui/cart/ProductList.tsx

Layout de grid

```
grid grid-cols-1 lg:grid-cols-3
// Lista de ítems: lg:col-span-2
// Panel de resumen: lg:col-span-1 lg:sticky lg:top-20
```

Estado vacío (Empty State)

Antes: <Link><button> (HTML inválido) con texto plano. Después: patrón 'empty state' estándar en e-commerce:

```
<div className="text-center py-16">
  <ShoppingBagIcon className="mx-auto h-16 w-16 text-gray-300" />
  <h3 className="mt-4 text-lg font-semibold">Tu carrito está vacío</h3>
  <p className="text-gray-500">Agrega productos para continuar.</p>
  <Link href="/dashboard" className="mt-4 inline-block ...">
    Ver productos
  </Link>
</div>
```

Botón Vaciar carrito

Antes: bg-blue-500, texto 'Limpiar carrito'. Después: enlace de texto text-red-500 hover:text-red-700. En e-commerce, 'vaciar carrito' es una acción destructiva secundaria; no debe tener el mismo peso visual que 'Confirmar compra'.

PARTE 2 — IMPLEMENTACIÓN RESPONSIVE

2.1 Principios y estrategia aplicada

Se utilizó el enfoque mobile-first de Tailwind CSS:

- **Estilos sin prefijo:** aplican a TODOS los tamaños (incluido mobile).
- **Prefijos sm:, md:, lg::** añaden o sobrescriben estilos para pantallas más grandes.

Breakpoints utilizados

Prefijo	Ancho mínimo	Dispositivo objetivo
sm:	640px	Tablets pequeñas / landscape de móviles
md:	768px	Tablets
lg:	1024px	Laptops y escritorio
xl:	1280px	Desktop grande (no usado en este proyecto)

Criterio principal de diseño mobile

'¿Este elemento cabe en 375px (iPhone estándar) sin desbordar ni verse aplastado?' Antes de cada decisión de layout, se evalúa el caso de 375px como peor escenario razonable (hay iPhones con 320px, pero el soporte para esa anchura es opcional en proyectos modernos).

2.2 NavBar en mobile

Problema

Con una sesión activa, el navbar mostraba: 'Productos' + 'Carrito' + 'Mis compras' + 'Logout'. En mobile (375px) con texto en cada botón, el total superaba el ancho disponible.

Solución — texto oculto en mobile

```
<span className="hidden sm:inline">Carrito</span>
<span className="hidden sm:inline">Mis compras</span>
```

En mobile el botón muestra SOLO el ícono (w-5 h-5, ~44px). El ícono aumentó de w-4 a w-5 para ser más táctil (zona de toque mínima recomendada por Material Design: 44×44px).

Resultado en mobile

- **Con sesión activa:** [IAW.]   [Logout]
- **Sin sesión:** [IAW.]  [Login]

El link 'Productos' en mobile

```
// Solo visible en sm+
<Link href="/dashboard" className="hidden sm:flex ...">
  Productos
</Link>
```

El logo 'IAW.' ya funciona como home/branding en mobile. El link 'Productos' es redundante en pantallas pequeñas y ocupa espacio valioso.

2.3 Formularios Login y Register en mobile

Problema

El formulario tenía p-8 (32px de padding en cada lado). En un iPhone SE (320px de ancho), esto consume 64px solo en padding horizontal, dejando 256px para el contenido.

Solución

```
// Padding responsivo
p-5 sm:p-8
// p-5 = 20px padding en mobile → 280px de contenido en 320px
// p-8 = 32px padding en sm+ → diseño original en tablet/desktop
```

El panel de marca derecho usa hidden lg:flex. En mobile el formulario ocupa el 100% del ancho de pantalla con bg-gray-50 como fondo, diferenciando la card blanca del fondo.

2.4 ProductDetails en mobile

Problema

aspect-square en mobile generaba una imagen de 375×375px (casi pantalla completa en vertical) antes de llegar al contenido del producto.

Solución — aspect ratio diferenciado por breakpoint

```
// aspect-[4/3] en mobile, aspect-square en sm+
className="relative aspect-[4/3] sm:aspect-square"
```

- En mobile (375px): imagen de 375×281px → 94px más corta que aspect-square.
- En sm+: vuelve al cuadrado (1:1) original.
- Tailwind soporta aspect ratios arbitrarios con la sintaxis aspect-[w/h].

Tipografía reducida en mobile

```
h1:      text-xl  sm:text-2xl
precio:  text-2xl sm:text-3xl
```

2.5 Carrito (ProductList) en mobile

Layout de grid

```
grid-cols-1 lg:grid-cols-3
```

- En mobile (< 1024px): ítems y resumen se apilan verticalmente.
- En desktop (lg+): ítems en 2/3 del ancho, resumen en 1/3 con sticky.
- El resumen no hace sticky en mobile: el usuario necesita ver todos los ítems antes de confirmar.

Padding responsivo

```
px-4 sm:px-6 // Horizontal: más estrecho en mobile
py-4 sm:py-6 // Vertical: menos espacio en mobile
gap-4 sm:gap-6 // Separación entre ítems: más compacta en mobile
```

2.6 Overflow horizontal — diagnóstico y solución

Síntoma

Scroll horizontal en mobile en las páginas dashboard y landing.

Diagnóstico

Las páginas tenían divs con max-w-7xl mx-auto SIN px-4:

```
// ❌ Sin padding: puede causar overflow en mobile
<div className="mx-auto max-w-7xl">
<div className="mx-auto max-w-7xl py-4">
```

max-w-7xl (1280px) en mobile actúa como width: 100% del viewport. Sin px-4, el contenido hijo se renderizaba desde el pixel 0, llegando exactamente al borde del viewport. Elementos con min-width implícito (inputs, imágenes) podían exceder el viewport por pocos píxeles.

Solución A — Corrección estructural

```
// ✅ Con padding horizontal seguro
<div className="mx-auto max-w-7xl px-4">
<div className="mx-auto max-w-7xl px-4 py-4">
```

px-4 = 16px cada lado → 32px totales de margen horizontal seguro.

Solución B — Barrera de seguridad en globals.css

```
/* globals.css */
```

```
html,
body {
  overflow-x: hidden;
}
```

⚠ Atención: *overflow-x: hidden en el body puede interferir con position: sticky en algunos navegadores. En este proyecto no hay sticky en el body directo, sino en el nav y en el panel del carrito, que están dentro del flujo normal y no se ven afectados.*

2.7 Tipografía fluida

Se aplicó el patrón de escala tipográfica responsive para evitar que los títulos grandes se recorten o generen overflow en mobile:

Componente / Elemento	Mobile	Desktop (sm+)
dashboard/page.tsx — h1	text-2xl	sm:text-4xl
HomePage.tsx — h2	text-xl	sm:text-3xl
ProductDetails — h1	text-xl	sm:text-2xl
ProductDetails — precio	text-2xl	sm:text-3xl

Patrón general

```
text-{tamaño-mobile} sm:text-{tamaño-desktop}
```

El tamaño mobile es siempre un escalón por debajo del desktop.

Tipografía fluida con clamp() (avanzado)

Tailwind no tiene tipografía fluida nativa. Para implementación avanzada con CSS puro se puede usar la función clamp():

```
/* Escala de 20px (mobile) a 36px (desktop) entre 320px y 1280px */
font-size: clamp(1.25rem, 1.25rem + 1.5vw, 2.25rem);
```

Para usar esto en Tailwind, se extiende el tema con un plugin o se usa la clase arbitrary [font-size:clamp(1.25rem,1.25rem+1.5vw,2.25rem)]. El patrón de breakpoints es suficiente para la mayoría de proyectos.

RESUMEN DE ARCHIVOS MODIFICADOS

Archivo	Cambio principal
src/app/globals.css	overflow-x: hidden en html y body
src/app/layout.tsx	metadata, lang='es', viewport
src/app/page.tsx	Hero section, CTAs, semantica
src/app/dashboard/page.tsx	px-4 al max-w-7xl, tipografia
src/app/login/layout.tsx	Split layout con gradiente de marca
src/app/register/layout.tsx	Idem login layout
src/app/login/LoginForm.tsx	Inputs styled, padding responsive
src/app/register/page.tsx	Idem LoginForm
src/app/ui/shared/NavBar.tsx	<nav>, sticky, responsive
src/app/ui/shared/CartButton.tsx	Icono + texto hidden sm:inline
src/app/ui/shared/BuysRecordButton.tsx	Idem CartButton
src/app/ui/shared/LoginButton.tsx	Link-as-button
src/app/ui/shared/LogoutButton.tsx	button nativo con signOut
src/app/ui/shared/HomePage.tsx	px-4, tipografia responsive
src/app/ui/dashboard/Cards.tsx	Card con shadow, Image fill
src/app/ui/dashboard/SearchBar.tsx	focus-within, MagnifyingGlassIcon
src/app/ui/dashboard/Pagination.tsx	Estilos de boton, iconos, espanol
src/app/ui/dashboard/AddToCartButton.tsx	Backdrop, animacion, CheckCircleIcon
src/app/ui/dashboard/ProductDetails.tsx	Link volver, fill image, responsive
src/app/ui/cart/CartCard.tsx	Iconos, rounded-xl imagen
src/app/ui/cart/ProductList.tsx	Grid responsive, empty state, resumen

APÉNDICE — CONCEPTOS CLAVE PARA EL EXAMEN

A.1 Resumen de patrones y técnicas

Técnica	Definición breve	Cuándo usarla
link-as-button	Estilos de botón en elemento <a> o <Link>	Cuando el destino es una URL

Técnica	Definición breve	Cuándo usarla
focus-within	Estilos al padre cuando un hijo tiene foco	Inputs agrupados con botón
mobile-first	Estilos base para mobile, se amplían con breakpoints	Siempre con Tailwind CSS
fill + object-cover	Imagen que llena su contenedor sin distorsión	Imágenes de cards con ratio fijo
hidden sm:inline	Oculto en mobile, visible en sm+	Texto de botones de navbar
empty state	UI cuando no hay datos: ícono + título + CTA	Carrito vacío, lista sin resultados
sticky resumen	Panel de resumen lg:sticky lg:top-20	Carrito desktop, sidebars

A.2 Reglas de accesibilidad aplicadas

- Todo botón que solo muestra un ícono debe tener aria-label con descripción textual de la acción.
- La barra de navegación debe usar la etiqueta semántica <nav> con aria-label.
- Los elementos interactivos (<a>, <button>) no deben anidarse entre sí.
- Los colores de texto sobre fondo deben cumplir un ratio de contraste mínimo de 4.5:1 (WCAG AA).
- Los modales deben tener un backdrop oscuro para comunicar el estado de bloqueo del fondo.

A.3 Glosario de clases Tailwind clave

Clase	Efecto
overflow-hidden	Recorta el contenido que desborda el contenedor (necesario con border-radius)
transition-shadow	Anima solo la propiedad box-shadow (más performante que transition-all)
truncate	Aplica overflow: hidden + text-overflow: ellipsis + white-space: nowrap
shrink-0	flex-shrink: 0. El elemento no se encoge aunque el flex container sea pequeño
inset-0	top: 0; right: 0; bottom: 0; left: 0. Cubre todo el contenedor posicionado
aspect-[4/3]	Ratio de aspecto arbitrario 4:3 (Tailwind v3+)
ring-offset-1	Espacio blanco entre el elemento y el anillo de foco

Clase	Efecto
bg-black/50	Negro con 50% de opacidad (sintaxis de opacidad en Tailwind v3)